



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**título del TFG
Documentación Técnica**



Presentado por nombre alumno
en Universidad de Burgos — 28 de enero
de 2026

Tutor: nombre tutor

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	15
Apéndice B Especificación de Requisitos	17
B.1. Introducción	17
B.2. Objetivos generales	17
B.3. Catálogo de requisitos	17
B.4. Especificación de requisitos	21
Apéndice C Especificación de diseño	43
C.1. Introducción	43
C.2. Diseño de datos	43
C.3. Diseño arquitectónico	44
C.4. Diseño procedimental	44
C.5. Diseño de interfaces	44
Apéndice D Documentación técnica de programación	53
D.1. Introducción	53
D.2. Estructura de directorios	53

D.3. Manual del programador	53
D.4. Compilación, instalación y ejecución del proyecto	53
D.5. Pruebas del sistema	53
Apéndice E Documentación de usuario	55
E.1. Introducción	55
E.2. Requisitos de usuarios	55
E.3. Instalación	55
E.4. Manual del usuario	55
Apéndice F Anexo de sostenibilización curricular	57
F.1. Introducción	57
Bibliografía	59

Índice de figuras

A.1. <i>Burndown chart</i> del <i>sprint</i> 1.	4
A.2. <i>Burndown chart</i> del <i>sprint</i> 2.	5
A.3. <i>Burndown chart</i> del <i>sprint</i> 3.	5
A.4. <i>Burndown chart</i> del <i>sprint</i> 4.	6
A.5. <i>Burndown chart</i> del <i>sprint</i> 5.	8
A.6. <i>Burndown chart</i> del <i>sprint</i> 6.	9
A.7. <i>Burndown chart</i> del <i>sprint</i> 7.	9
A.8. <i>Burndown chart</i> del <i>sprint</i> 8.	11
A.9. <i>Burndown chart</i> del <i>sprint</i> 9.	12
A.10. <i>Burndown chart</i> del <i>sprint</i> 10.	13
A.11. <i>Burndown chart</i> del <i>sprint</i> 11.	14
A.12. <i>Burndown chart</i> del <i>sprint</i> 12.	15
A.13. <i>Burndown chart</i> del <i>sprint</i> 13.	16
 B.1. Diagrama de casos de uso.	 25
C.1. Diagrama Entidad/Relación.	45
C.2. Diagrama relacional.	46
C.3. Prototipo de la página de inicio.	47
C.4. Prototipo de la página de inicio de sesión.	47
C.5. Prototipo de la página de registro.	48
C.6. Prototipo de la página principal.	48
C.7. Prototipo de la página del historial.	49
C.8. Prototipo de los detalles del historial.	49
C.9. Prototipo de la página de consulta.	50
C.10. Prototipo de la página de gestión de usuarios.	50
C.11. Prototipo de la página de añadir usuarios.	51
C.12. Prototipo de la página de edición del usuario.	51

C.13.Prototipo de la página de gestión de documentos.	52
---	----

Índice de tablas

A.1. Tiempo estimado de los <i>Story Points</i>	2
B.1. CU-1 Registro de usuarios.	23
B.2. CU-2 Iniciar sesión.	24
B.3. CU-3 Cerrar sesión.	26
B.4. CU-4 Realizar consulta al <i>RAG</i>	27
B.5. CU-5 Consultar historial de consultas.	28
B.6. CU-6 Borrar consultas del historial.	29
B.7. CU-7 Cargar documentos de licitaciones.	30
B.8. CU-8 Importar licitaciones automáticamente.	31
B.9. CU-9 Procesar documentos cargados.	32
B.10.CU-10 Indexar documentos.	33
B.11.CU-11 Actualizar la base de datos vectorial.	34
B.12.CU-12 Listar documentos.	35
B.13.CU-13 Consultar estado de los documentos.	36
B.14.CU-14 Eliminar documento.	37
B.15.CU-15 Gestión de usuarios.	38
B.16.CU-16 Crear usuarios.	39
B.17.CU-17 Borrar usuarios.	40
B.18.CU-18 Modificar rol del usuarios.	41
B.19.CU-19 Editar usuario.	42

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este documento define cómo se va a gestionar y ejecutar el desarrollo de un producto o sistema informático. Establece el alcance, los objetivos, el cronograma, los recursos, el presupuesto, los riesgos y los criterios de calidad, así como la organización del equipo y los mecanismos de seguimiento y control. En esencia, convierte una idea de software en un itinerario gestionable, con responsabilidades claras y procedimientos para controlar el progreso y los cambios.

Su importancia radica en que reduce la incertidumbre y aumenta la probabilidad de éxito académico y técnico. Permite justificar decisiones, anticipar problemas, y demostrar competencias en gestión de proyectos, ingeniería de requisitos, calidad y comunicación técnica.

En este documento se va a incluir la planificación temporal de las fases y tareas del proyecto. Para cada una se debe establecer una fecha de inicio y se debe estimar la fecha final. Para realizar esto hay que tener en cuenta el peso de cada tarea, los requisitos previos y las dependencias.

También, se va a valorar la viabilidad del proyecto. En concreto, la viabilidad económica y la legal. En la viabilidad económica se estiman los costes y los beneficios del proyecto. Mientras que en la viabilidad legal analizan las leyes y licencias que afecten al desarrollo.

A.2. Planificación temporal

Para la planificación se utiliza una metodología Scrum, aplicando iteraciones (*sprints*). Al finalizar cada *sprint* se realiza una reunión para la revisión del incremento y la planificación del siguiente. En la planificación se seleccionan y valoran las tareas (*issues*) a realizar en el siguiente *sprint*. Estas tareas se han valorado usando los *story points* de ZenHub. A las tareas que se subdividen en otras se les ha asignado solo puntos extra respecto a sus subtareas, los *story points* no se han considerado acumulativos.

Para simplificar la asignación de los *story points* se les ha asignado una estimación temporal recogida en la tabla A.1:

<i>Story Points</i>	Tiempo estimado (horas)
1	0,5
2	1
3	2
5	4
8	6
13	9
21	15
40	más de 20

Tabla A.1: Tiempo estimado de los *Story Points*.

Sprint 0 (11/09/2025 - 18/09/2025)

Este *sprint* marcó el comienzo del proyecto. Los objetivos fueron realizar una primera toma de contacto con los *RAG* y los *LLM*, investigando sobre qué son, cómo funcionan y qué ventajas aportan los *RAG*. También, se planteó empezar a escribir los conceptos teóricos relevantes y crear el repositorio **GitHub** para el proyecto. Como el *sprint* se diseñó antes de la creación del repositorio, no se creó el *milestone* asociado como se ha hecho en los *sprints* posteriores. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Crear el repositorio en GitHub.
- Investigar sobre los *RAG* y *LLM*.
 - Leer el capítulo 1 del libro: *A simple guide to retrieval augmented generation* [1].

- Leer el capítulo 2 del libro: *A simple guide to retrieval augmented generation* [1]
- Escribir conceptos teóricos sobre lo investigado.

Para realizar estas tareas se estimaron 6 horas de trabajo pero en realidad fueron 9. Al finalizar el *sprint* se completaron todas las tareas.

Sprint 1 (18/09/2025 - 25/09/2025)

Los objetivos del **Sprint 1** fueron investigar sobre los *LLM* y los *RAG* y escribir la información más relevante como conceptos teóricos en la memoria. Comenzar a escribir los objetivos del proyecto y las técnicas y herramientas definidas hasta el momento, como la metodología Scrum o el repositorio. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Escribir objetivos del proyecto y técnicas y herramientas.
- Escribir conceptos teóricos.
 - Investigar sobre los *RAG* y los *LLM*.
 - Leer el capítulo 3 del libro: *A simple guide to retrieval augmented generation* [1].
 - Leer el capítulo 4 del libro: *A simple guide to retrieval augmented generation* [1].

Para realizar estas tareas se estimaron 8,5 horas de trabajo pero en realidad fueron 10. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura **A.1**

Sprint 2 (25/09/2025 - 02/10/2025)

Los objetivos del **Sprint 2** fueron seguir investigando sobre los *LLM* y *RAG* y explicar la información relevante en los conceptos teóricos. Investigar sobre el *Web Scraping* y explicar en los anexos el plan de proyecto, en concreto la planificación temporal y los *sprints*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Añadir las correcciones a la memoria.
- Completar el apartado *pipeline* de generación del apartado conceptos teóricos.
- Investigar sobre *Web Scraping*.



Figura A.1: *Burndown chart* del *sprint* 1.

- Investigar sobre HTML.
- Investigar sobre CSS.
- Explicar el plan de proyectos en los anexos.

Para realizar estas tareas se estimaron 10 horas de trabajo pero en realidad fueron 12. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.2

***Sprint* 3 (2/10/2025 - 09/10/2025)**

Los objetivos del **Sprint 3** fueron realizar un primer prototipo de *Web Scraping*, tanto en *Selenium* como en *Playwright*. Incluir en la memoria las correcciones y la información relevante. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Añadir las correcciones a la memoria.
- Realización del comienzo de la aplicación de *web scraping*.
 - Investigar *web scraping* con Selenium.
 - Investigar *web scraping* con Playwright.
- Escribir conceptos teóricos en la memoria.
 - Leer el capítulo 5 del libro: *A simple guide to retrieval augmented generation* [1].

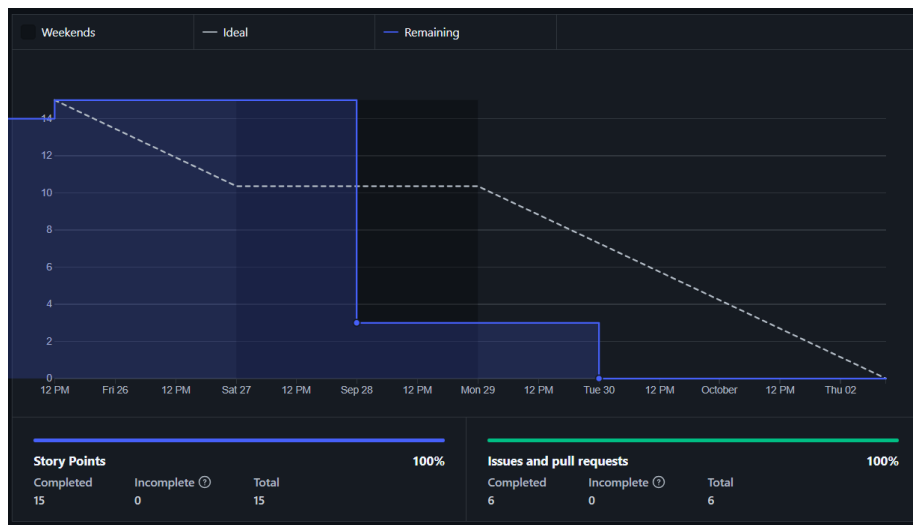


Figura A.2: *Burndown chart del sprint 2.*



Figura A.3: *Burndown chart del sprint 3.*

Para realizar estas tareas se estimaron 15 horas de trabajo pero en realidad fueron 18. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura [A.3](#)

Figura A.4: *Burndown chart* del *sprint* 4.

***Sprint* 4 (9/10/2025 - 16/10/2025)**

Los objetivos del **Sprint** 4 fueron continuar realizando la aplicación de *Web Scraping*, tanto en *Selenium* como en *Playwright*, para sacar licitaciones de la página de contratación del sector público. Incluir en la memoria una comparación entre ambas aplicaciones de *Web Scraping*. Seguir investigando sobre los *RAG*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Continuar con el *script* de *web scraping* usando Playwright.
 - Continuar con el *script* de *web scraping* usando Selenium.
 - Estudiar sobre json.
- Escribir conceptos teóricos en la memoria.
 - Leer el capítulo 6 del libro: *A simple guide to retrieval augmented generation* [1].

Para realizar estas tareas se estimaron 14 horas de trabajo pero en realidad fueron 18. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.4

Sprint 5 (16/10/2025 - 23/10/2025)

Los objetivos del **Sprint 5** fueron investigar sobre las mejoras que aporta la *API* asíncrona de *Playwright* y como se implementa. También, se planificado investigar sobre la gestión de dependencias y entornos en *Python*. Todo esto mientras se sigue investigando sobre los *RAG* y *LLM* y se escriben los aspectos más relevantes en la memoria. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Realizar código con *Playwright* asíncrono.
 - Investigar sobre la *API* asíncrona de *Playwright*.
- Investigar sobre las dependencias y los entornos en *Python*.
- Escribir conceptos teóricos en la memoria.
 - Leer más capítulo del libro: *A simple guide to retrieval augmented generation* [1].

Para realizar estas tareas se estimaron 15 horas de trabajo pero en realidad fueron 18, ya que se tuvo que reescribir parte del código realizado en los *sprints* anteriores para el *Web Scraping* debido a una actualización en la página de **Contratación del Sector Público**. Este cambio afectó en mayor medida a la extracción de información de las licitaciones, ya que ha sido la página que más ha cambiado con dicha actualización. A pesar de este contratiempo al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura **A.5**

Sprint 6 (23/10/2025 - 30/10/2025)

Los objetivos del **Sprint 6** fueron descargar los documentos (en formato *pdf*) de las licitaciones de la página de **Contratación del Sector Público**. Para descargar dichos *pdfs* se va a usar de base el programa de *Web Scraping* del *sprint* anterior. Además, se han fijado como objetivos investigar sobre los modelos de inteligencia artificial de *Hugging Face* e instalar y probar el funcionamiento de *Ollama*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Investigar sobre *Ollama*.
- Investigar y probar modelos de inteligencia artificial de *Hugging Face*.
- Investigar sobre formas de implementar los *RAG*.
- Descargar los *pdfs* de las licitaciones de la la página de **Contratación del Sector Público**.



Figura A.5: *Burndown chart* del *sprint* 5.

- Instalar las dependencias en *Poetry*.

Para realizar estas tareas se estimaron 12 horas de trabajo pero en realidad fueron 15, se ha usado un servidor de la UBU para descargar un zip con las licitaciones. Para acceder a dichas licitaciones se ha usado el archivo «*resultados_playwright_asincrono*» obtenido en el *sprint* anterior, en el cual se almacenan los enlaces a los pdfs de cada licitación. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura [A.6](#)

***Sprint* 7 (30/10/2025 - 6/11/2025)**

El objetivo principal del **Sprint 7** fue investigar e implementar distintos tipos de *tokenizer*. Esto se planteó como un primer paso para la implantación total del *RAG*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Investigar sobre los *tokenize*.
- Implementar distintos tipos de *tokenize*.
- Comparar las implementaciones de *tokenize*.

Para realizar estas tareas se estimaron 12 horas de trabajo pero en realidad fueron 15. En este *sprint* se han llevado a cabo todas las tareas



Figura A.6: *Burndown chart* del *sprint* 6.



Figura A.7: *Burndown chart* del *sprint* 7.

planificadas, pero sería destacable que en los *scripts* de las diversas pruebas de *tokenizers* no solo se han incluido estos. Si no que también se han incluido funciones para probar su uso, como pueden ser *prompts* y funciones para dividir los pdfs de las licitaciones.

El *burndown* de este *sprint* se ilustra en la figura [A.7](#)

Sprint 8 (6/11/2025 - 13/11/2025)

El objetivo principal del **Sprint 8** fue implementar una base de datos para el *RAG* y juntarlo con uno de los *tokenizers* ya implementados del *sprint* anterior. Además, en este *sprint* se planteó como objetivo obtener los pliegos definitivos de de la página de **Contratación del Sector Público**.

Esto se debe a que los pliegos obtenidos anteriormente no eran correctos, se trataban de resúmenes incompletos que no servían para alimentar al *RAG*. Durante el *sprint 7*, se encontraron los pliegos correctos y se fijó como objetivo del siguiente *sprint* modificar los programa de *Web Scraping* para descargarlos. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Obtener pliegos correctos.
- Estudiar sobre las distintas bases de datos.
- Implementar la base de datos.
- Explicar la comparativa de las bases de datos.

Para realizar estas tareas se estimaron 12 horas de trabajo, pero en realidad fueron 16. En este *sprint* se han llevado a cabo todas las tareas planificadas. Se ha implementado la base de datos vectorial *Qdranty* y se ha juntado con el diseño de *tokenizer* del *sprint* anterior, aunque se ha cambiado el modelo para usar el del libro ***LLM Engineer's Handbook***. Además, se han realizado los *embeddings* para poder almacenarlos en la base de datos vectorial.

El *burndown* de este *sprint* se ilustra en la figura **A.8**

Sprint 9 (13/11/2025 - 20/11/2025)

El objetivo principal del **Sprint 9** fue realizar un *script* que, dada una consulta, devuelva el *chunk* más parecido. Junto con este *chunk* debe devolver el nombre y el título del pdf, así como una respuesta del *LLM* basándose en dicho *chunk*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Implementar funciones que devuelvan el nombre del pdf al que pertenece el *chunk*.
- Mejorar la partición de los *chunks* añadiendo solapamiento (*overlap*).
- Obtención del fragmento del *chunk*.



Figura A.8: *Burndown chart* del *sprint* 8.

- Implementar código para que el *LLM* de una respuesta a partir de una pregunta del usuario.
- Añadir al código funciones para medir los tiempos
- Usar loggers en el código

Para realizar estas tareas se estimaron 18 horas y más o menos se ha cumplido dicha estimación. En este *sprint* se han llevado a cabo todas las tareas planificadas. Se ha mejorado el código añadiendo medidas de los tiempos de ejecución e implementando el uso de *loggers*. También, se ha añadido solapamiento en los *chunks* lo que mejora la eficacia de estos. Por otro lado se ha implementado el objetivo principal, es decir, el *script* que permite que el usuario realice una pregunta y el *LLM* conteste basándose en el *chunk* más parecido a la pregunta. Además se indica el pdf del que ha sacado la información así como el *chunk* concreto.

El *burndown* de este *sprint* se ilustra en la figura A.9

***Sprint* 10 (20/11/2025 - 27/11/2025)**

El objetivo principal del **Sprint 10** fue intentar convertir los pdfs a *Markdown*. Para ello se han valorado diversas formas de realizar la conversión, por ejemplo, desde un *OCR* vinculado a un *LLM* o directamente empleando una biblioteca de *Python*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

Figura A.9: *Burndown chart* del *sprint 9*.

- Buscar formas de automatizar la conversión a *Markdown*.
- Realizar la conversión de un pdf a *Markdown*.
 - Automatizar la conversión a *Markdown*.

Para realizar estas tareas se estimaron 10 horas, pero debido a la gran heterogeneidad de formatos de los pdfs fueron más, aproximadamente unas 15. A pesar de que se han obtenido resultados significativamente mejores que en las primeras aproximaciones no se ha conseguido ningún **Markdown** lo suficiente mente bueno. Esto se debe a que cada pdf de licitaciones tiene un formato distinto y no se ha conseguido ningún *Script* que detecte correctamente todas las secciones.

El *burndown* de este *sprint* se ilustra en la figura [A.10](#)

Sprint 11 (27/11/2025 - 4/12/2025)

El objetivo principal del **Sprint 11** fue intentar implementar un *Script* que basándose en los resultados obtenidos del anterior sea capaz de detectar correctamente las secciones de los pdfs. Además, se propuso comenzar a investigar sobre el funcionamiento de *Flask*. Ya que esta aplicación se va a utilizar en el proyecto para realizar una *web* para realizar las consultas al *LLM*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Probar nuevos OCRs para la conversión de los pdfs a *Markdown*.



Figura A.10: *Burndown chart* del *sprint* 10.

- Investigar sobre Flask.

Para realizar estas tareas se estimaron 8 horas. Al final se emplearon cerca de 12 horas debido al mismo problema del *Sprint* anterior, que no se ha conseguido implementar un *script* capaz de detectar correctamente todas las secciones.

El *burndown* de este *sprint* se ilustra en la figura A.11

***Sprint* 12 (4/12/2025 - 15/01/2026)**

El objetivo principal del **Sprint 12** fue comenzar a realizar la página *web* con *Flask* y comenzar a definir los casos de uso del proyecto. Este *sprint* fue más largo que el resto ya que coincidió con la semana de exámenes finales del primer cuatrimestre y las navidades. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Definir los casos de uso.
 - Identificar los casos de uso del proyecto.
 - Identificar los requisitos del proyecto.
- Comenzar la página *web*.
- Hacer la gestión básica de usuarios.
- Implementar la gestión de usuarios para los administradores.

Figura A.11: *Burndown chart* del *sprint* 11.

- Hacer los *test* para la gestión de usuarios de la *web*.

Para realizar estas tareas se estimaron 28 horas de trabajo. Al final se emplearon más de 35 horas, debido principalmente a que la tarea «Comenzar la página *web*» llevó más tiempo del planificado. Esta tarea consistió en realizar la estructura básica de la *web*, la cual finalmente se dividió en *blueprints* para que tuviese mejor escalabilidad. Comprender el funcionamiento de los *blueprints* y saber implementarlos aumentó significativamente el tiempo de desarrollo de la estructura de la *web*.

El *burndown* de este *sprint* se ilustra en la figura [A.12](#)

***Sprint* 13 (15/01/2025 - 22/01/2026)**

El objetivo principal del **Sprint 13** fue completar las funcionalidades de la *web* permitiendo que los administradores puedan cargar documentos, de forma manual o automática (mediante *web scraping*). El sistema gestionará estos documentos para extraer de ellos los *embeddings* necesarios para el *RAG*. Además, se marcó como objetivo incluir la interfaz para realizar las consultas al *LLM*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Añadir casos de uso.
- Incluir una clase consulta.



Figura A.12: *Burndown chart* del *sprint* 12.

- Añadir la funcionalidad carga de documentos.
- Mostrar historial de consultas.
- Implementar las consultas en la *web Flask*.
- Juntar el *web scraping* con la carga de documentos de la *web*.
- Realizar la interfaz de las consultas.
- Poner la clave secreta como variable de entorno.

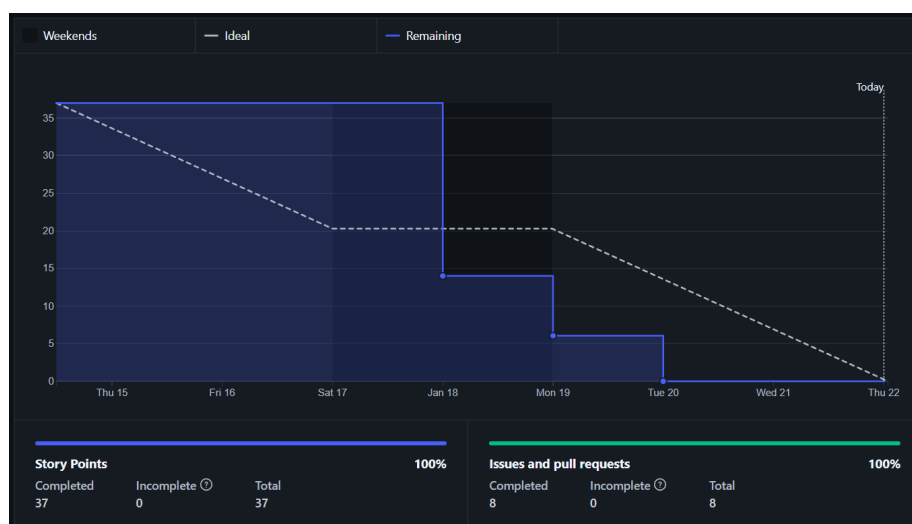
Para realizar estas tareas se estimaron 28,5 horas de trabajo. Al final del *sprint* se cumplieron todas las tareas más o menos en el plazo esperado.

El *burndown* de este *sprint* se ilustra en la figura [A.13](#)

A.3. Estudio de viabilidad

Viabilidad económica

Viabilidad legal

Figura A.13: *Burndown chart* del sprint 13.

Apéndice B

Especificación de Requisitos

B.1. Introducción

B.2. Objetivos generales

Los objetivos generales que se buscan cumplir con este proyecto son:

- Desarrollar un sistema basado en Generación Aumentada por Recuperación (*RAG*) que permita realizar consultas sobre licitaciones del sector público, enriqueciendo las respuestas mediante información documental relevante.
- Automatizar la recopilación, procesamiento e indexación de documentos de licitaciones, de forma que puedan ser integrados eficientemente en una base de datos vectorial.
- Facilitar la consulta de la información de las licitaciones mediante una aplicación web intuitiva y accesible para usuarios autenticados.
- Mejorar la comprensión y análisis de la información administrativa y técnica contenida en las licitaciones.

B.3. Catálogo de requisitos

A continuación se van a definir los requisitos del proyecto, tanto los funcionales como los no funcionales.

Requisitos funcionales

- RF-1 Gestión de documentos: el sistema debe permitir que el administrador gestione los documentos de licitaciones del sistema.
 - RF-1.1 Inserción de documentos: el sistema debe permitir cargar documentos de licitaciones con el objetivo de construir y actualizar la base de datos vectorial.
 - RF-1.2 Listado de documentos cargados: el sistema debe permitir ver los documentos cargados y sus metadatos.
 - RF-1.3 Borrado de documentos: el sistema debe permitir borrar los documentos cargados.
- RF-2 Procesado de los documentos: el sistema debe procesar de manera automática los documentos introducidos para prepararlos para que el *RAG* pueda usarlos.
 - RF-2.1 Extracción de información: el sistema debe extraer automáticamente el contenido textual de los documentos introducidos, ignorando páginas o documentos sin texto extraíble.
 - RF-2.2 Conversión del contenido: el sistema debe transformar el texto extraído a un formato homogéneo, concretamente a *markdown* para que sea más fácil su procesamiento.
 - RF-2.2.1 Conservar estructura básica: el sistema debe conservar la estructura básica del texto siempre que sea posible (secciones, títulos, saltos de línea, párrafos, etc).
 - RF-2.3 Limpieza y normalización del texto: el sistema debe limpiar y normalizar el texto que se extrae de los documentos.
 - RF-2.3.1 Eliminar caracteres: el sistema debe eliminar los caracteres irrelevantes o problemáticos.
 - RF-2.3.2 Normalización: el sistema debe normalizar los espacios, saltos de línea y formatos inconsistentes.
- RF-3 Segmentación en *chunks*: el sistema debe dividir los documentos ya procesados en fragmentos de texto (*chunks*) adecuados para el modelo de *embeddings*.
 - RF-3.1 Tamaño: el sistema debe segmentar el texto respetando un máximo de tamaño por *token* para evitar errores.
 - RF-3.2 Solapamiento: el sistema debe aplicar solapamiento entre *chunks* para preservar el contexto.
 - RF-3.3 Contexto: cada *chunk* debe tener una referencia al documento original y a su posición dentro de él.
- RF-4 Generación de *embeddings*: el sistema debe generar representaciones vectoriales (*embeddings*) para cada *chunk*.

- RF-5 Persistencia en base de datos vectorial: el sistema debe almacenar los *chunks* y sus *embeddings* en una base de datos vectorial.
 - RF-5.1 Almacenar *chunks*: el sistema debe almacenar el contenido textual de cada *chunk*.
 - RF-5.2 Almacenar *embeddings*: el sistema debe almacenar el *embedding* asociado a cada *chunk*.
 - RF-5.3 Almacenar metadatos: el sistema debe almacenar metadatos relevantes como el nombre del archivo, el título y el segmento incluido en el *chunk*.
 - RF-5.4 Actualización: el sistema debe permitir la actualización o recreación de la colección vectorial en el caso de incluir documentos nuevos.
 - RF-5.5 Eliminar: el sistema debe permitir borrar los *chunks* y *embeddings* de la base de datos vectorial.
- RF-6 Recuperación de información por similitud: el sistema debe recuperar información relevante a partir de las consultas de los usuarios.
 - RF-6.1 Generación de *embedding*: el sistema debe generar el *embedding* de la consulta del usuario.
 - RF-6.2 Búsqueda por similitud: el sistema debe realizar una búsqueda por similitud en la base de datos vectorial.
- RF-7 Construcción del *prompt* con contexto: el sistema debe construir un *prompt*, para alimentar al *LLM*, que incluya los *chunks* recuperados, la pregunta del usuario y reglas de uso del contexto.
- RF-8 Generación de respuestas mediante un *LLM*: el sistema debe enviar el *prompt* generado a un modelo *LLM* y mostrar la respuesta de dicho modelo al usuario.
- RF-9 Trazabilidad de respuesta: el sistema debe devolver junto con la respuesta los metadatos del documento origen y los *chunks* empleados para la generación de la respuesta.
- RF-10 Gestión de usuarios: el sistema debe permitir la gestión de usuarios a través de la aplicación web.
 - RF-10.1 Registros y autenticación: el sistema debe permitir el registro y autenticación de los usuarios.
 - RF-10.2 Sesiones: el sistema debe gestionar las sesiones de los usuarios de forma segura.
 - RF-10.3 Roles: el sistema debe permitir asignar a los usuarios el rol de administradores o usuarios normales.
 - RF-10.4 Administradores: los administradores deben poder añadir usuarios, eliminarlos y cambiar su rol.

- RF-10.5 Edición de datos de usuario: los usuarios deben poder editar sus datos.
- RF-11 Interfaz de consultas: los usuarios autenticados podrán realizar consultas y ver las respuestas del sistema.
 - RF-11.1 Historial de consultas: los usuarios podrán ver su historial de consultas.
 - RF-11.2 Borrar consultas: los usuarios podrán borrar consultas del historial.
- RF-12 Importación automática de licitaciones: el sistema debe permitir al administrador importar de manera automática las licitaciones mediante *web scraping*.
 - RF-12.1 Descarga y almacenamiento: el sistema debe descargar y almacenar los documentos obtenidos, así como sus metadatos.
 - RF-12.2 Gestión de errores y duplicados: el sistema debe continuar aunque la descarga de algún documento falle. Debe registrar los errores y evitar duplicados.

Requisitos no funcionales

- RNF-1 Reproducibilidad: el proyecto debe ejecutarse en un entorno reproducible.
- RNF-2 Rendimiento: el sistema debe ser capaz de procesar los documentos sin intervención manual. Además, el sistema tiene que responder en un tiempo razonable.
- RNF-3 Robustez ante errores: el sistema debe continuar ejecutándose ante fallos esperables en datos, red o servicios externos.
 - RNF-3.1 Documentos corruptos: el sistema debe saltarse los documentos corruptos o sin texto extraíble y continuar con el resto de documentos.
 - RNF-3.2 *Timeouts*: el sistema debe manejar los *timeouts* del *LLM* devolviendo mensajes controlados al usuario.
- RNF-4 Trazabilidad técnica: el sistema debe realizar *logs* de carga de modelo, lectura de PDFs, *chunking*, *embeddings*, guardado y consulta.
- RNF-5 Seguridad: el sistema debe gestionar de forma segura la información sensible de los usuarios y el acceso a la aplicación.
 - RNF-5.1 Contraseñas: las contraseñas deben almacenarse *hasheadas*.

- RNF-5.2 Control de acceso por roles: las funciones administrativas deben estar protegidas por rol para que solo las pueda realizar el administrador.
- RNF-5.3 Protección de secretos: las claves sensibles no deben estar públicas en el repositorio.
- RNF-6 Usabilidad: la interfaz debe permitir a los usuarios realizar las consultas y ver el historial de manera sencilla. Además, los textos deben ser legibles.
- RNF-7 Calidad del *software*: debe existir un conjunto de pruebas que asegure el correcto funcionamiento del programa.
- RNF-8 Disponibilidad: la web debe garantizar un nivel de disponibilidad adecuado que permita a los usuarios acceder a la aplicación y realizar consultas.
- RNF-9 Recuperación y tolerancia a fallos: el sistema debe disponer de mecanismos que permitan la recuperación de información y la restauración del servicio tras fallos, garantizando la integridad de los datos.
- RNF-10 Evitar alucinaciones: el sistema no debe responder cuando no haya contexto relevante suficiente.

B.4. Especificación de requisitos

Los actores del sistema son:

- **Usuario anónimo**: usuario que no ha iniciado sesión (solo puede registrarse y autenticarse)
- **Usuario autenticado**: usuario que puede acceder a la aplicación sus acciones en al aplicación cambiarán dependiendo de su rol.
 - **Usuario normal**: usuario autenticado que puede consultar el *RAG* y ver su historial
 - **Usuario administrador**: usuario autenticado con permisos de gestión de usuarios y de carga y actualización documental
- **Sistema**: es el encargado del procesamiento, *chunking*, *embeddings*, guardado y consulta.

Los casos de uso definidos para el proyecto son:

- CU-1 Registro de usuarios
- CU-2 Iniciar sesión

- CU-3 Cerrar sesión
- CU-4 Realizar consulta al *RAG*
- CU-5 Consultar historial de consultas
- CU-6 Borrar consultas del historial
- CU-7 Cargar documentos de licitaciones
- CU-8 Importar licitaciones automáticamente
- CU-9 Procesar documentos cargados
- CU-10 Indexar documentos
- CU-11 Actualizar la base de datos vectorial
- CU-12 Listar documentos
- CU-13 Consultar estado de los documentos
- CU-14 Eliminar documento
- CU-15 Gestión de usuarios
- CU-16 Crear usuarios
- CU-17 Borrar usuarios
- CU-18 Modificar rol del usuario
- CU-19 Editar usuario

El diagrama de los casos de uso se muestra en esta imagen [B.1](#).

CU-1	Registro de usuarios
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.1
Descripción	Permite crear una cuenta en la aplicación web.
Precondición	Que el usuario no este autenticado.
Acciones	1. El usuario accede a «Registro». 2. El usuario introduce el nombre, email y contraseña. 3. El sistema valida el formato y que el email no este duplicado. 4. El sistema almacena el usuario con la contraseña <i>hasheada</i>. 5. El sistema confirma el registro. 6. El sistema crea la sesión y redirige a la página de consultas.
Postcondición	Cuenta creada y usuario autenticado
Excepciones	1. Tiene algún campo vacío. Muestra un error con el mensaje «Completa este campo». 2. El email ya existe. Muestra un error con el mensaje «Ya existe un usuario con ese email». 3. El email no cumple la validación. Muestra un error con el mensaje «Invalid email address». 4. La contraseña no cumple la política. Muestra un error con el mensaje «Aumenta la longitud del texto a 6 caracteres como mínimo». 5. Las contraseñas no coinciden. Muestra un error con el mensaje «Las contraseñas no coinciden». 6. El nombre tiene menos de 2 caracteres. Muestra un error con el mensaje «Aumenta la longitud del texto a 2 caracteres o más».
Importancia	Alta

Tabla B.1: CU-1 Registro de usuarios.

CU-2	Iniciar sesión
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.1, RF-10.2
Descripción	Autentifica al usuario y abre una sesión segura.
Precondición	Que el usuario este registrado y no este autenticado.
Acciones	1. El usuario accede a «Iniciar sesión». 2. El usuario introduce el email y la contraseña. 3. El sistema verifica los credenciales. 4. El sistema crea la sesión y redirige a la página de consultas.
Postcondición	Sesión activa para el usuario.
Excepciones	1. Tiene algún campo vacío. Muestra un error con el mensaje «Completa este campo». 2. El email o la contraseña no coinciden con ningún usuario registrado. Muestra un error con el mensaje «Email o contraseña incorrectos.». 3. El email no cumple la validación. Muestra un error con el mensaje «Invalid email address». 4. La contraseña no cumple la política. Muestra un error con el mensaje «Aumenta la longitud del texto a 6 caracteres como mínimo».
Importancia	Alta

Tabla B.2: CU-2 Iniciar sesión.

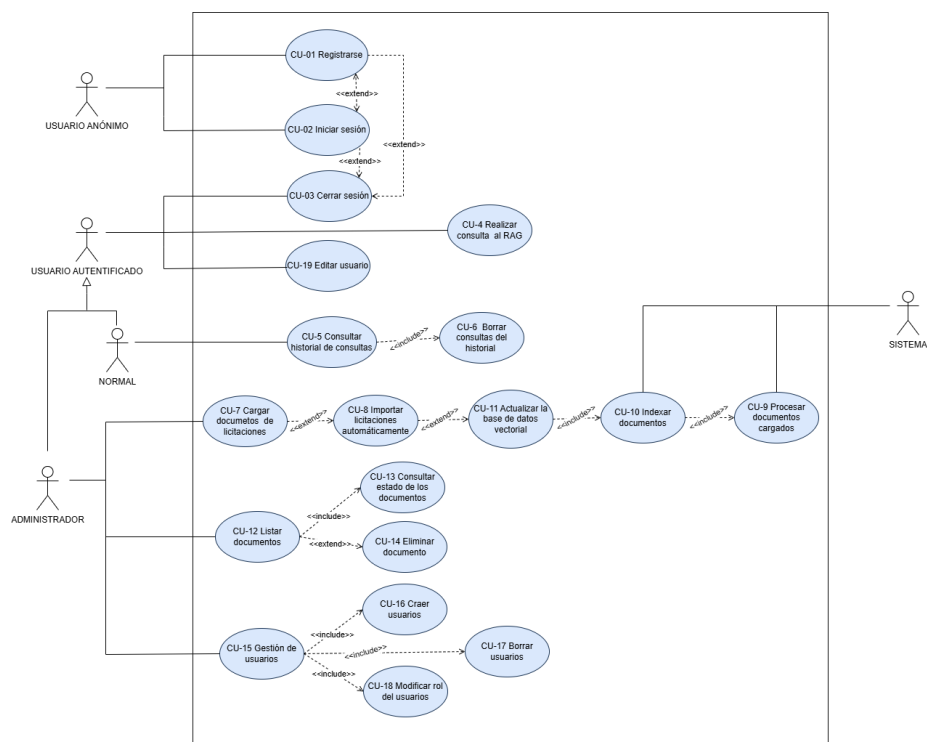


Figura B.1: Diagrama de casos de uso.

CU-3	Cerrar sesión
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.2
Descripción	Finaliza la sesión del usuario.
Precondición	Que el usuario este autenticado.
Acciones	1. El usuario pulsa el botón «Cerrar sesión». 2. El sistema invalida la sesión. 3. El sistema redirige a la primera de inicio.
Postcondición	Sesión finalizada.
Excepciones	1. Error al invalidar sesión. Muestra un mensaje de error controlado.
Importancia	Media

Tabla B.3: CU-3 Cerrar sesión.

CU-4	Realizar consulta al <i>RAG</i>
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-6, RF-7, RF-8, RF-9, RF-11, RNF-3, RNF-3.2, RNF-10
Descripción	El usuario hace una pregunta, el sistema recupera contexto por similitud, construye el prompt y genera respuesta que incluya las fuentes.
Precondición	Que el usuario este autenticado y la base de datos vectorial este disponible.
Acciones	1. El usuario escribe y envía una pregunta. 2. El sistema genera <i>embeddings</i> de la consulta. 3. El sistema busca por similitud en la base de datos vectorial. 4. El sistema construye un <i>prompt</i> con los <i>chunks</i> y las reglas de uso del contexto. 5. El sistema llama al <i>LLM</i> y genera respuestas. 6. El sistema llama al <i>LLM</i> y genera una respuesta. 7. El sistema adjunta la trazabilidad de la respuesta, la cual incluye los documentos, metadatos y <i>chunks</i> utilizados. 8. El sistema guarda la consulta en el historial del consultas del usuario.
Postcondición	Respuesta mostrada y consulta registrada.
Excepciones	1. Cuando no hay contexto suficiente. El sistema no responde a la pregunta (para evitar alucinaciones) y muestra un aviso de que falta contexto. 2. <i>Timeout</i> del <i>LLM</i>. El sistema muestra un mensaje controlado.
Importancia	Alta

Tabla B.4: CU-4 Realizar consulta al *RAG*.

CU-5	Consultar historial de consultas
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-11.1
Descripción	Permite al usuario ver su lista de preguntas y respuestas anteriores.
Precondición	Que el usuario este autenticado y existan consultas guardadas.
Acciones	1. El usuario accede al «Historial». 2. El sistema lista las consultas del usuario. 3. El usuario puede abrir una consulta para ver la respuesta completa y la trazabilidad de dicha respuesta.
Postcondición	Historial visualizado.
Excepciones	1. El usuario no ha realizado consultas. Muestra un mensaje de error «Aún no hay consultas».
Importancia	Alta

Tabla B.5: CU-5 Consultar historial de consultas.

CU-6	Borrar consultas del historial
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-11.2
Descripción	Permite al usuario borrar consultas del historial.
Precondición	Que el usuario este autenticado y existan consultas guardadas.
Acciones	1. El usuario accede al «Historial». 2. El sistema lista las consultas del usuario. 3. El usuario pulsa el botón «Borrar». 4. El sistema elimina la consulta seleccionada de la base de datos. 5. El sistema actualiza la vista del historial.
Postcondición	La consulta seleccionada se elimina del historial del usuario.
Excepciones	1. Error en la base de datos al intentar borrar la consulta. Muestra un mensaje de error controlado. 2. La consulta seleccionada no existe o ya ha sido borrada. Muestra un aviso al usuario.
Importancia	Alta

Tabla B.6: CU-6 Borrar consultas del historial.

CU-7	Cargar documentos de licitaciones
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1, RF-10.3, RF-10.4, RNF-3, RNF-3.1
Descripción	El administrador sube documentos para alimentar o ampliar la base de datos vectorial.
Precondición	Que el administrador este autenticado.
Acciones	1. El administrador pulsa el botón «Cargar». 2. El administrador selecciona uno o varios documentos. 3. El sistema valida los documentos y los almacena. 4. El sistema registra la carga.
Postcondición	Los documentos están disponibles para procesado.
Excepciones	1. Documento corrupto o sin texto legible. Se marca como fallido y se continua con el siguiente.
Importancia	Alta

Tabla B.7: CU-7 Cargar documentos de licitaciones.

CU-8	Importar licitaciones automáticamente
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-12, RF-12.1, RF-12.2, RNF-3, RNF-3.1
Descripción	Permite al administrador importar de manera automática las licitaciones a través de <i>web scraping</i> .
Precondición	Que el administrador este autenticado y haya un programa de <i>web scraping</i> funcional.
Acciones	1. El administrador accede a «Administrar documentos». 2. El administrador pulsa el botón «Web scraping». 3. El sistema lanza el proceso de <i>web scraping</i>. 4. El sistema guarda los documentos obtenidos y registra sus metadatos. 5. El sistema muestra un resumen con el número de documentos descargados, los duplicados y los fallidos.
Postcondición	Los documentos importados están disponibles para procesado e indexado.
Excepciones	1. Fallo de red. Se registra y muestra al administrador. 2. Documento corrupto o sin texto legible. Se marca como fallido y se continua con el siguiente. 3. Documento duplicado. Se omite y se registra como duplicado
Importancia	Media

Tabla B.8: CU-8 Importar licitaciones automáticamente.

CU-9	Procesar documentos cargados
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-2, RF-2.1, RF-2.2, RF-2.2.1, RF-2.3, RF-2.3.1, RF-2.3.2, RNF-3, RNF-3.1
Descripción	El sistema prepara automáticamente los documentos para que puedan incluirse en el sistema.
Precondición	Existen documentos cargados pendientes.
Acciones	1. El sistema extrae texto descartando páginas y documentos sin texto extraíble. 2. Convierte el texto a <i>Markdown</i> homogéneo. 3. Intenta conservar la estructura básica del documento original. 4. Limpia caracteres problemáticos y normaliza espacios y saltos de línea.
Postcondición	Los documentos procesados listos para el <i>chunking</i> .
Excepciones	1. Documento sin texto. Se marca como no procesable y se continua como los demás.
Importancia	Alta

Tabla B.9: CU-9 Procesar documentos cargados.

CU-10	Indexar documentos
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-3, RF-4, RF-5, RF-5.1, RF-5.2, RF-5.3
Descripción	El sistema convierte los documentos procesados en <i>chunks</i> , genera los <i>embeddings</i> y los guarda en la base de datos vectorial con metadatos.
Precondición	Existen documentos procesados y una base de datos vectorial.
Acciones	1. El sistema segmenta los archivos en <i>chunks</i>. 2. Aplica solapamiento a dichos <i>chunks</i>. 3. Añade una referencia al documento así como el segmento que contiene. 4. Genera <i>embeddings</i> de cada <i>chunk</i>. 5. Lo almacena en la base de datos vectorial.
Postcondición	La colección vectorial con los nuevos documentos incluidos.
Excepciones	1. Fallo en conseguir los <i>embedding</i> de un <i>chunk</i>. Se omite el <i>chunk</i> y se continúa.
Importancia	Alta

Tabla B.10: CU-10 Indexar documentos.

CU-11	Actualizar la base de datos vectorial
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1, RF-5.4, RF-10.3
Descripción	Permite al administrador actualizar la base de datos vectorial cuando hay documentos nuevos.
Precondición	Hay un administrador autenticado y existen documentos nuevos que añadir a la base de datos vectorial.
Acciones	1. El administrador accede a «Administrar documentos». 2. El administrador pulsa el botón «Actualizar Base de Datos Vectorial». 3. El sistema realiza el <i>chunking</i> y el <i>embedding</i>. 4. El sistema lo almacena en la base de datos vectorial.
Postcondición	La colección vectorial esta actualizada.
Excepciones	1. Error de conexión con la base de datos vectorial. Se aborta la operación y se avisa al administrador a través de un mensaje.
Importancia	Alta

Tabla B.11: CU-11 Actualizar la base de datos vectorial.

CU-12	Listar documentos
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1.2, RNF-4
Descripción	Permite al administrador ver el listado de documentos junto con los metadatos relevantes.
Precondición	Hay un administrador autenticado y existe al menos un documento cargado en el sistema.
Acciones	1. El administrador accede a «Administrar documentos». 2. El sistema lista los documentos con sus metadatos (nombre, tamaño, fecha de modificación y <i>chunks</i>).
Postcondición	Documentos visibles para el administrador.
Excepciones	1. No hay documentos. Muestra un mensaje informativo «Aún no hay documentos». 2. Error de base de datos. Muestra un error.
Importancia	Media

Tabla B.12: CU-12 Listar documentos.

CU-13	Consultar estado de los documentos
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1.2, RNF-4
Descripción	Permite al administrador ver el estado de los documentos cargados en el sistema (cargado, procesado, indexado o fallido).
Precondición	Hay un administrador autenticado y existe al menos un documento cargado en el sistema.
Acciones	1. El administrador accede a «Administrar documentos». 2. El sistema lista los documentos con sus metadatos (nombre, tamaño, fecha de modificación y <i>chunks</i>). 3. El sistema muestra el estado de los documentos listados.
Postcondición	Estados de los documentos visibles para el administrador.
Excepciones	1. No hay documentos. Muestra un mensaje informativo «Aún no hay documentos». 2. Error de base de datos. Muestra un error.
Importancia	Media

Tabla B.13: CU-13 Consultar estado de los documentos.

CU-14	Eliminar documento
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1.3, RF-5.5, RNF-5.2
Descripción	Permite al administrador eliminar un documento del sistema, así como sus <i>chunks</i> y <i>embeddings</i> asociados en la base de datos vectorial.
Precondición	Hay un administrador autenticado y existe al menos un documento cargado en el sistema.
Acciones	1. El administrador accede a «Administrar documentos». 2. El sistema lista los documentos con sus metadatos (nombre, tamaño, fecha de modificación y <i>chunks</i>) y estado. 3. El administrador pulsa el botón «Borrar». 4. El sistema pide confirmación. 5. El administrador confirma el borrado. 6. El sistema borra el documento y sus metadatos. Además, si el documento esta indexado borra sus <i>chunks</i> y sus <i>embeddings</i>. 7. El sistema actualiza el listado.
Postcondición	Documento eliminado y sus <i>chunks</i> y <i>embeddings</i> borrados de la base de datos vectorial.
Excepciones	1. El documento no existe. Se muestra un aviso 2. Error al borra. No se borra y se informa al administrador. 3. Error al borrar en la base de datos vectorial. No se borra el documento y se informa al administrador.
Importancia	Alta

Tabla B.14: CU-14 Eliminar documento.

CU-15	Gestión de usuarios
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.3, RF-10.4
Descripción	El administrador debe poder gestionar a los usuarios de la aplicación y sus permisos.
Precondición	Hay un administrador autenticado.
Acciones	1. El administrador accede a la página de «Iniciar sesión». 2. El administrador introduce su <i>email</i> y contraseña. 3. El sistema le autentifica como administrador. 4. El administrador accede a «Administrar usuarios».
Postcondición	Listado de usuarios para editar.
Excepciones	1. Intento de acceso sin ser administrador. Se deniega. 2. Si no hay usuarios registrados en la aplicación. Muestra un mensaje.
Importancia	Media

Tabla B.15: CU-15 Gestión de usuarios.

CU-16	Crear usuarios
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.3, RF-10.4
Descripción	El administrador crea usuarios desde la web.
Precondición	Hay un administrador autenticado.
Acciones	1. El administrador accede a «Administrar usuarios». 2. El administrador pulsa el botón «Añadir usuario». 3. El administrador introduce el nombre, email, contraseña y si es administrador o no. 4. El sistema valida el formato y que el email no este duplicado. 5. El sistema almacena el usuario con la contraseña <i>hasheada</i>. 6. El sistema redirige a la página de «Gestión de usuarios».
Postcondición	Usuario creado.
Excepciones	1. Intento de acceso sin ser administrador. Se deniega. 2. Tiene algún campo vacío. Muestra un error con el mensaje «Completa este campo». 3. El email ya existe. Muestra un error con el mensaje «Ya existe un usuario con ese email». 4. El email no cumple la validación. Muestra un error con el mensaje «Invalid email address». 5. La contraseña no cumple la política. Muestra un error con el mensaje «Aumenta la longitud del texto a 6 caracteres como mínimo». 6. El nombre tiene menos de 2 caracteres. Muestra un error con el mensaje «Aumenta la longitud del texto a 2 caracteres o más».
Importancia	Media

Tabla B.16: CU-16 Crear usuarios.

CU-17	Borrar usuarios
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.3, RF-10.4
Descripción	El administrador borra usuarios desde la web.
Precondición	Hay un administrador autenticado y usuarios previamente registrados en la aplicación.
Acciones	1. El administrador accede a «Administrar usuarios». 2. El administrador pulsa el botón «Borra» del usuario que desee eliminar. 3. El sistema pide confirmación antes de borrar el usuario. 4. El administrador pulsa el botón «Aceptar». 5. El sistema borra al usuario y sus datos de la base de datos.
Postcondición	Usuario borrado.
Excepciones	1. Fallo al conectara con la base de datos. No se borra el usuario.
Importancia	Media

Tabla B.17: CU-17 Borrar usuarios.

CU-18	Modificar rol del usuarios
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.3, RF-10.4
Descripción	El administrador puede cambiar el rol de los usuarios desde la web.
Precondición	Hay un administrador autenticado y usuarios previamente registrados en la aplicación.
Acciones	1. El administrador accede a «Administrar usuarios». 2. El administrador pulsa el botón «Hacer admin»/«Quitar admin» del usuario que desee cambiar de rol. 3. El sistema cambia el rol del usuario.
Postcondición	Usuario con el rol cambiado.
Excepciones	1. Fallo al conectara con la base de datos. No se cmabia el rol del usuario.
Importancia	Baja

Tabla B.18: CU-18 Modificar rol del usuarios.

CU-19	Editar usuario
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.5, RNF-5.1, RNF-5.2
Descripción	Permite que un usuario edite sus datos y contraseña.
Precondición	Hay un usuario autenticado.
Acciones	1. El usuario accede a «Editar usuario». 2. Modifica los campos que desee (nombre, <i>email</i> o contraseña). 3. El sistema valida los formatos y reglas. 4. El sistema guarda los cambios. Si hay cambio de contraseña la almacena <i>hasheada</i>.
Postcondición	Datos del usuario actualizados.
Excepciones	1. El email ya existe. Muestra un error con el mensaje «Ya existe un usuario con ese email». 2. El email no cumple la validación. Muestra un error con el mensaje «Invalid email address». 3. La contraseña no cumple la política. Muestra un error con el mensaje «Aumenta la longitud del texto a 6 caracteres como mínimo». 4. El nombre tiene menos de 2 caracteres. Muestra un error con el mensaje «Aumenta la longitud del texto a 2 caracteres o más».
Importancia	Baja

Tabla B.19: CU-19 Editar usuario.

Apéndice C

Especificación de diseño

C.1. Introducción

C.2. Diseño de datos

Para llevar a cabo la aplicación se han definido las siguientes entidades:

- **Usuario:** esta entidad representa a la persona que usa la aplicación. Distingue entre usuario normal y administrador, al cual le permite realizar la gestión de documentos y usuarios. Esta entidad maneja un identificador (id) que es la clave primaria, un nombre, un email que debe ser único, una contraseña que se almacena *hasheada*, el último inicio de sesión y el rol.
- **Consulta:** esta entidad guarda las preguntas que realizan los usuarios, así como las respuestas que da el *LLM* alimentado por el *RAG*. Es la base para el historial de consultas donde se van a listar y borrar. Esta entidad maneja un identificador (id) que es la clave primaria, el identificador del usuario que realiza la consulta, la pregunta del usuario, la respuesta, los fragmentos utilizados, el tiempo de respuesta, y la fecha en la que se preguntó al modelo.
- **Documento:** esta entidad representa un pdf cargado en la aplicación. Maneja un identificador (id) como clave primaria, el nombre del fichero, el tamaño, la fecha de modificación, el número de *chunks* en la base de datos vectorial, el *hash* del documento y el estado del documento (cargado, procesado, indexado, fallido).
- **Chunk:** esta entidad representa cada trozo de texto en el que se dividen los documentos para poder generar los *embeddings*. Maneja el texto

que forma al propio *chunk*, el *embedding* y los metadatos del *chunk* (nombre del archivo, título y posición dentro del documento).

- *Embedding*: esta entidad se corresponde con la representación vectorial utilizada para la base de datos vectorial. Va a permitir recuperar *chunks* por similitud. La entidad maneja un vector asociado al *chunk*.
- *ConsultaCunk*: esta entidad modela la relación entre las consultas y los *chunks* permitiendo que una consulta tenga varios *chunk* y que un *chunk* pueda servir para varias consultas. La entidad maneja el identificador de la consulta, el identificador del *chunk*, la similitud y el *ranking*.

Las relaciones entre las entidades son:

- Un usuario puede hacer cero o varias consultas.
- Cada consulta pertenece a un usuario.
- Un documento tiene cero (si esta cargado pero no indexado) o varios *chunks*.
- Cada *chunk* pertenece a un documento.
- Un *chunk* puede tener cero o un *embedding*.
- Cada *embedding* pertenece a un *chunk*.
- Una consulta usa uno o varios *chunk*.
- Un *chunk* puede ser usado por cero o varias consultas.

El diagrama entidad relación se muestra en esta imagen [C.1](#)

El diagrama relacional se muestra en esta imagen [C.2](#)

C.3. Diseño arquitectónico

C.4. Diseño procedimental

C.5. Diseño de interfaces

El diseño de las interfaces se ha desarrollado de manera incremental. Inicialmente se hicieron bocetos genéricos para plasmar las principales funcionalidades de la aplicación. Posteriormente se fue iterando sobre esos diseños hasta llegar a las interfaces finales de la *web*.

Los prototipos iniciales que se crearon se ilustran en las figuras [C.3](#), [C.4](#), [C.5](#), [C.6](#), [C.7](#), [C.8](#), [C.9](#), [C.10](#), [C.11](#), [C.12](#) y [C.13](#).

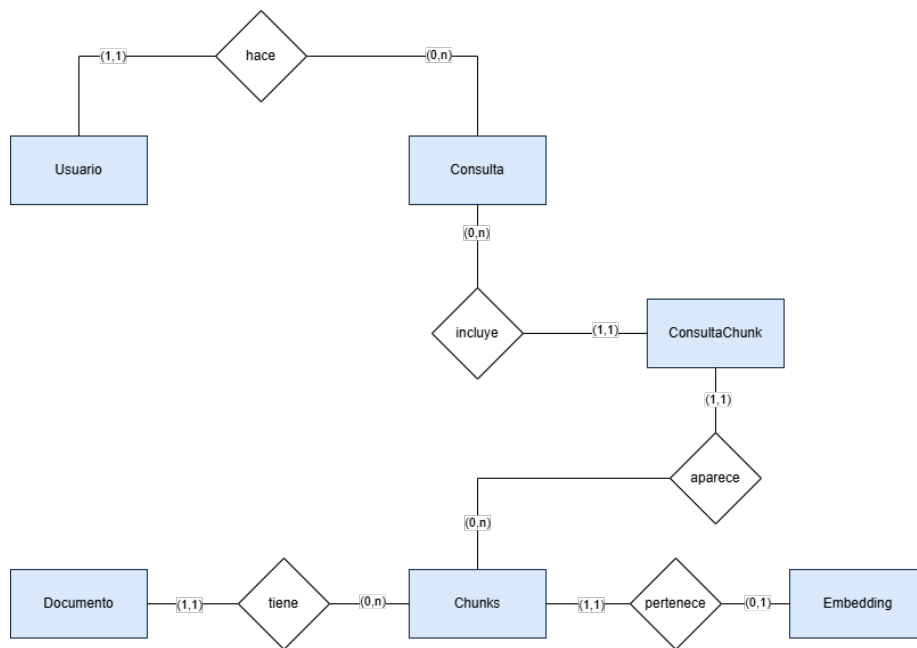


Figura C.1: Diagrama Entidad/Relación.

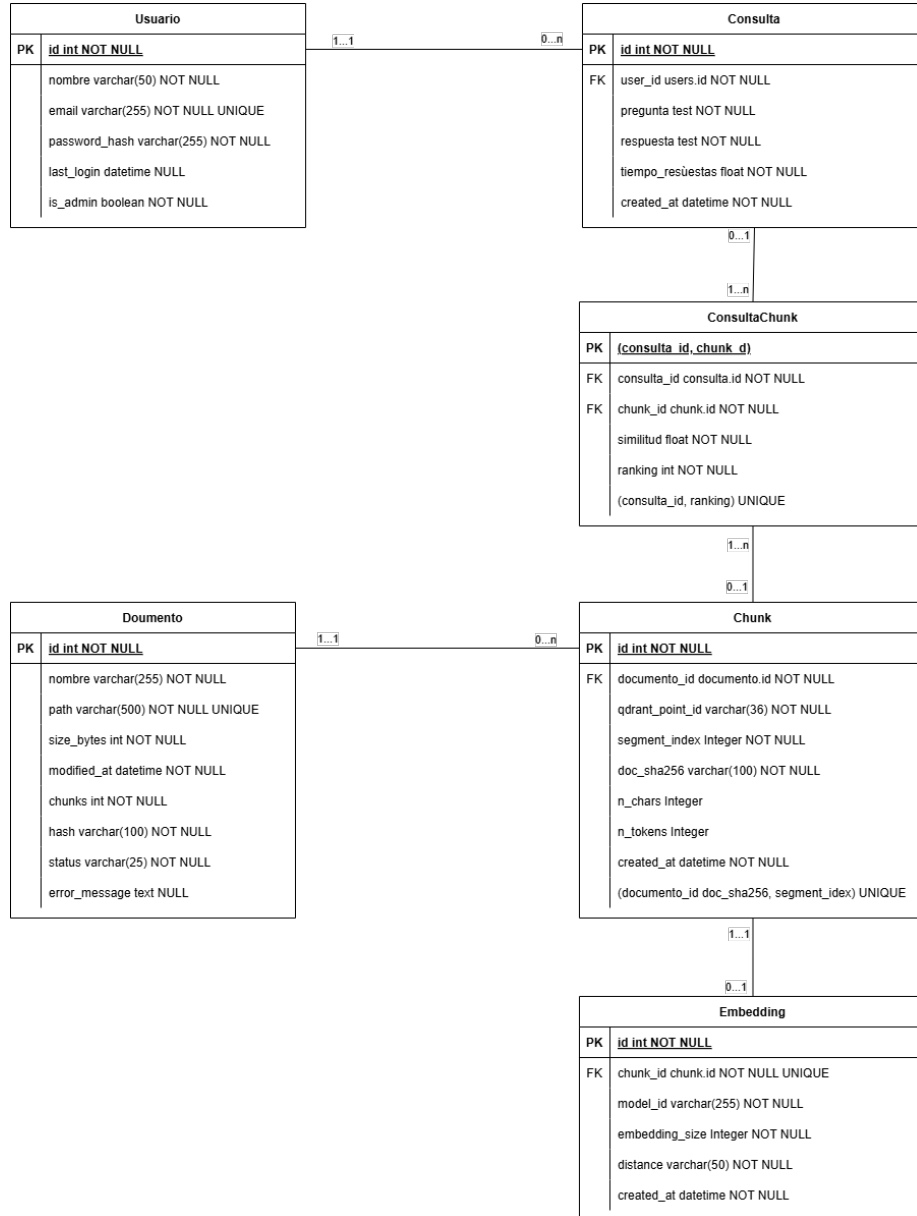


Figura C.2: Diagrama relacional.

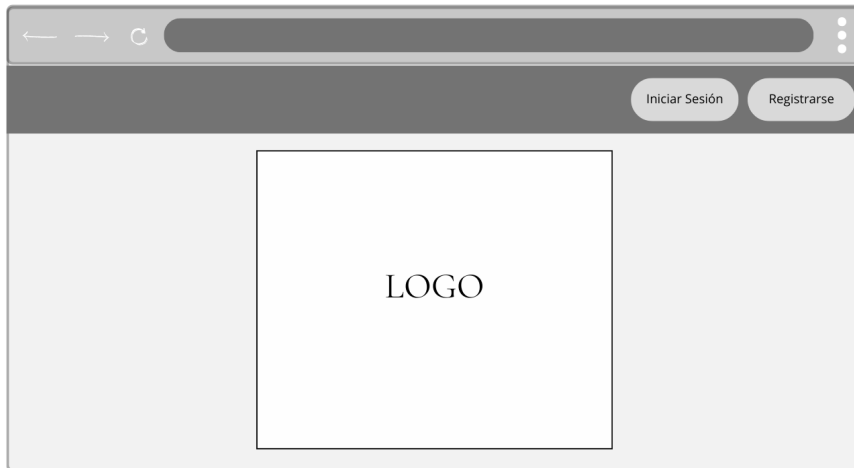


Figura C.3: Prototipo de la página de inicio.

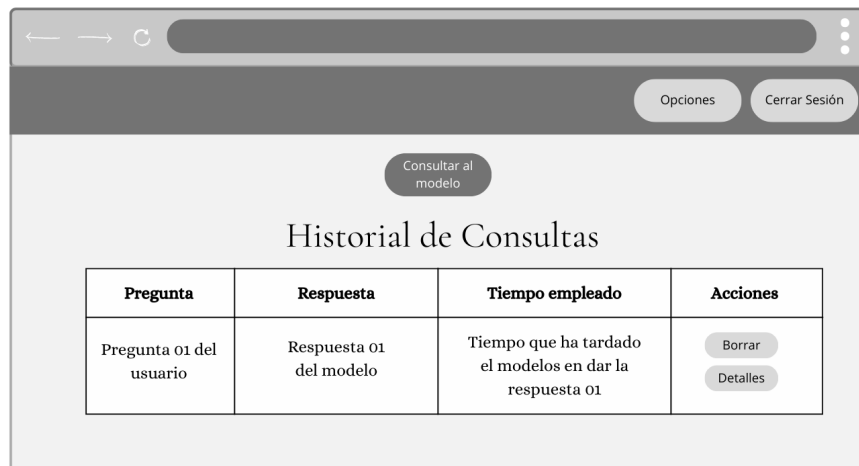


Figura C.4: Prototipo de la página de inicio de sesión.



Prototipo de la página de registro. La interfaz muestra un formulario con los campos: Nombre, Email, Contraseña y Repite la contraseña. Debajo de los campos hay dos botones: Registrarse e Iniciar Sesión.

Figura C.5: Prototipo de la página de registro.



Prototipo de la página principal. La interfaz muestra un botón "Consultar al modelo" y una tabla con el título "Historial de Consultas".

Pregunta	Respuesta	Tiempo empleado	Acciones
Pregunta 01 del usuario	Respuesta 01 del modelo	Tiempo que ha tardado el modelos en dar la respuesta 01	Borrar Detalles

Figura C.6: Prototipo de la página principal.

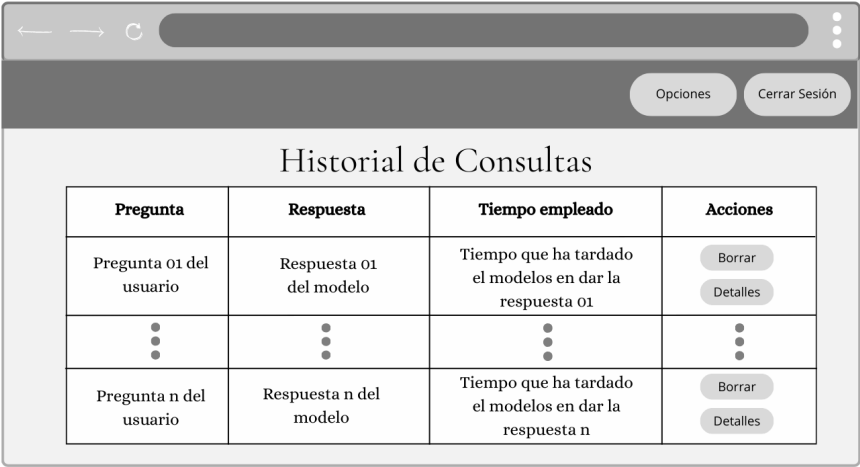


Figura C.7: Prototipo de la página del historial.

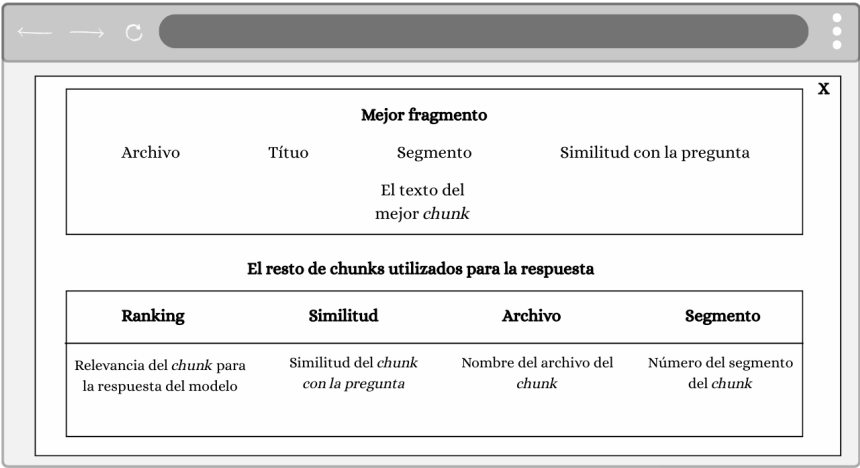


Figura C.8: Prototipo de los detalles del historial.

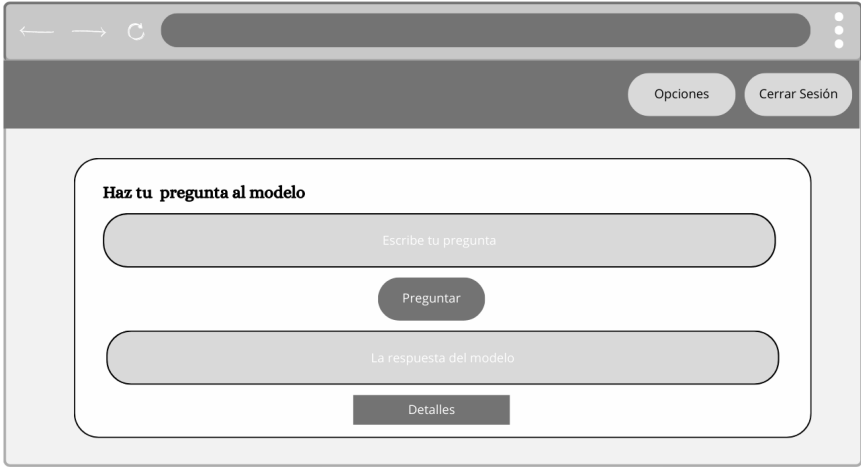


Figura C.9: Prototipo de la página de consulta.

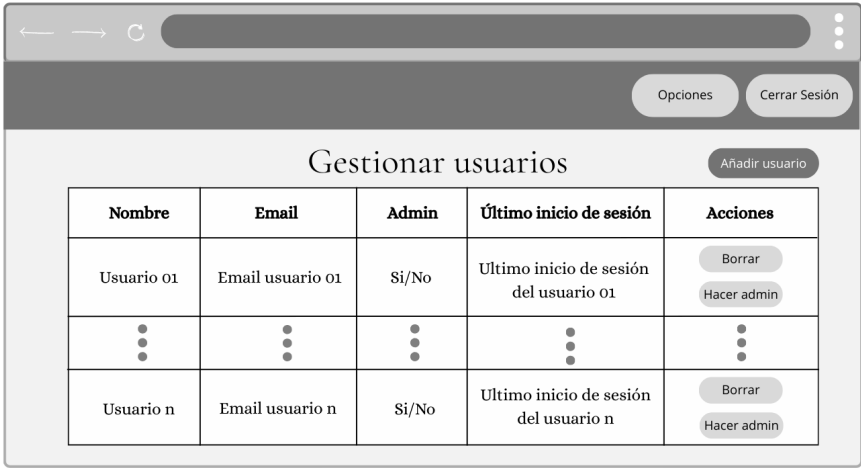


Figura C.10: Prototipo de la página de gestión de usuarios.

Prototipo de la página de añadir usuarios. La interfaz muestra un navegador web con una barra de direcciones y botones de navegación. El contenido principal es un formulario centrado con el título "Añadir usuario". El formulario contiene tres campos de texto etiquetados "Nombre", "Email" y "Contraseña". Debajo de estos campos hay un checkbox etiquetado "¿Es administrador?". Al final del formulario hay un botón redondeado etiquetado "Añadir usuario".

Figura C.11: Prototipo de la página de añadir usuarios.

Prototipo de la página de edición del usuario. La interfaz muestra un navegador web con una barra de direcciones y botones de navegación. El contenido principal es un formulario centrado con el título "Editar Usuario". El formulario contiene tres campos de texto etiquetados "Nombre", "Email" y "Contraseña". Los campos "Nombre" y "Email" tienen el texto "Nombre del usuario" y "Email del usuario" respectivamente. Al final del formulario hay un botón redondeado etiquetado "Editar".

Figura C.12: Prototipo de la página de edición del usuario.

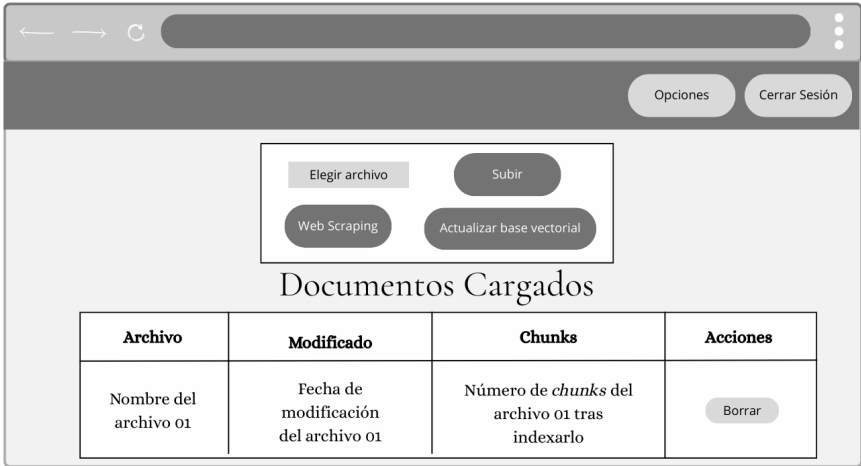


Figura C.13: Prototipo de la página de gestión de documentos.

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Bibliografía

- [1] Abhinav Kimothi. *A Simple Guide to Retrieval Augmented Generation*. Manning Publications, July 2025. [Consultado 8-Septiembre-2025 a 26-Septiembre-2025].